

Jeremiah Richard

Professor Jeff Lei

CSE-4317-001

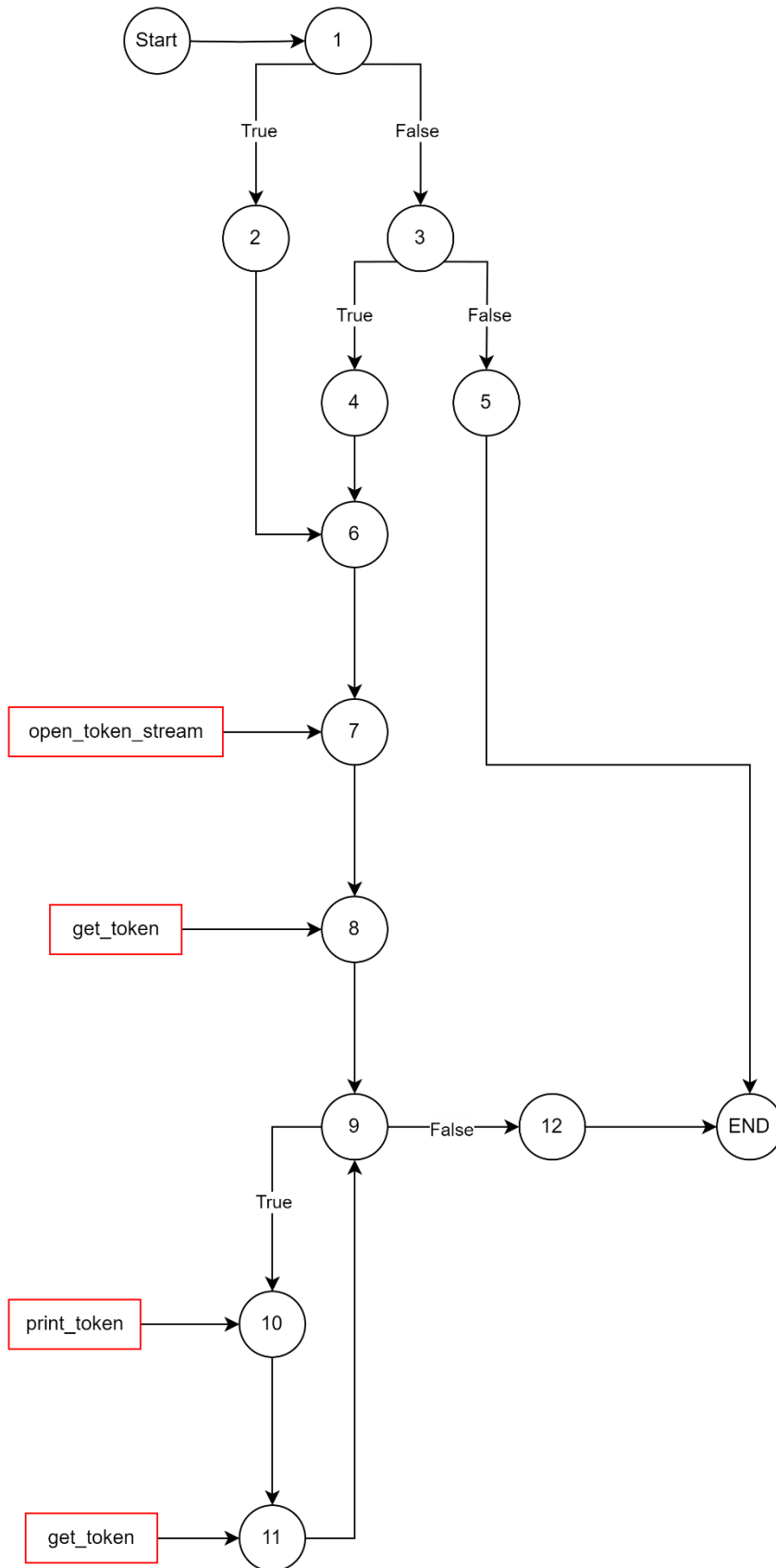
7/6/2023

Project

MAIN

```
458 public static void main(String[] args) {
459     String fname = null;
460     if (args.length == 0) { /* if not given filename, take as "" */
461         fname = new String();
462     } else if (args.length == 1) {
463         fname = args[0];
464     } else {
465         System.out.print("Error! Please give the token stream\n");
466         System.exit(0);
467     }
468     Printtokens t = new Printtokens();
469     BufferedReader br = t.open_token_stream(fname); /* open token stream */
470     String tok = t.get_token(br);
471     while (tok != null) { /* take one token each time until eof */
472         t.print_token(tok);
473         tok = t.get_token(br);
474     }
475 }
476
477 System.exit(0);
478 }
479 }
```

Block Number	Lines	Entry	Exit	Function Calls
1	459, 460	459	460	
2	461	461	461	
3	462	462	462	
4	463	463	463	
5	465, 466	465	466	
6	468	468	468	
7	469	469	469	open_token_stream
8	470	470	470	get_token
9	471	471	471	
10	472	472	472	print_token
11	473	473	473	get_token
12	477	477	477	



EDGE-COVERAGE

TEST PATHS:

(1, 2, 6, 7, 8, 9, 10, 12, END),

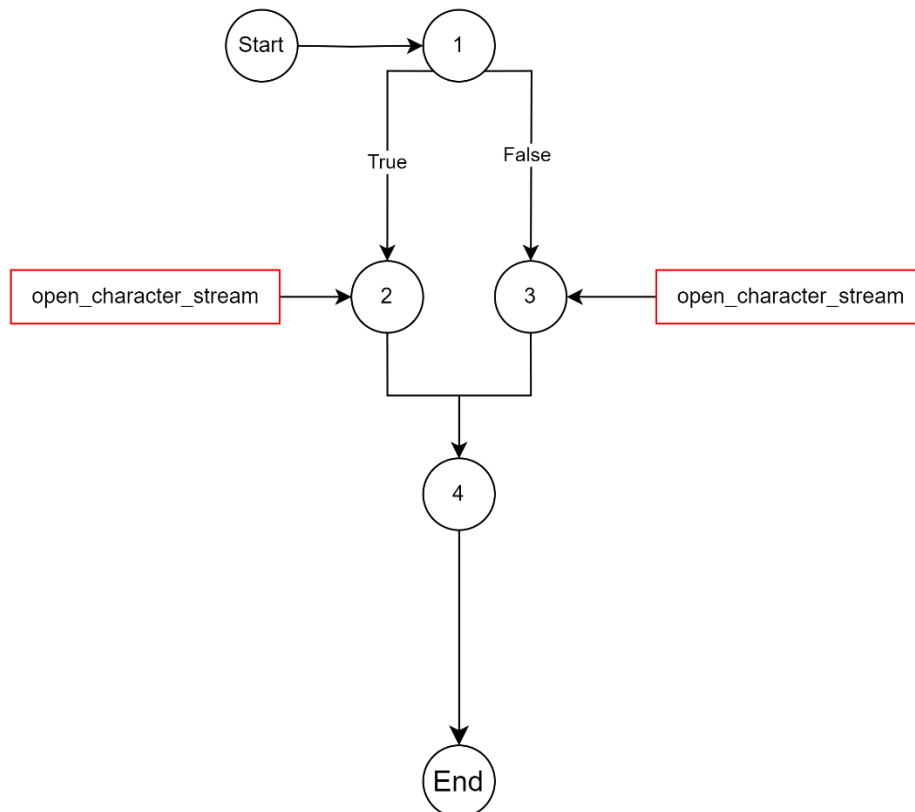
(1, 3, 4, 6, 7, 8, 9, 12, END),

(1, 5, END)

OPEN_TOKEN_STREAM

```
77 BufferedReader open_token_stream(String fname)
78 {
79     BufferedReader br;
80     if(fname==null || fname.equals(""))
81         br=open_character_stream(null);
82     else
83         br=open_character_stream(fname);
84     return br;
85 }
86
```

Block Number	Lines	Entry	Exit	Function Calls
1	79, 80	79	80	
2	81	81	81	open_character_stream
3	83	83	83	open_character_stream
4	84	84	84	



EDGE-COVERAGE

TEST PATHS:

```
[  
t1: (1, 2, 4),  
t2: (1, 2, 3)  
]
```

Unit Test

T1

- Input: String fname = "testfile.txt"
- Output: "RANDOM STUFF // ##\n"

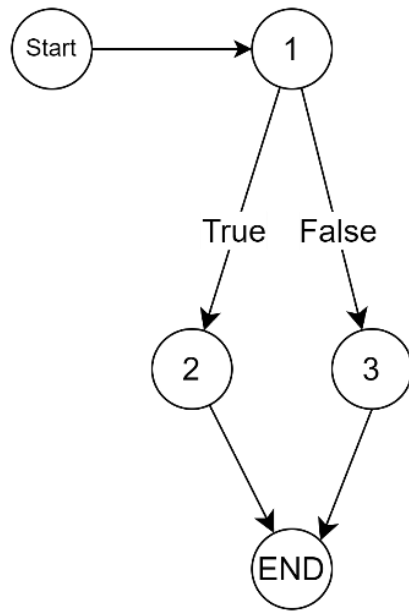
T2

- Input: String fname = ""
- Output: NULL

OPEN_CHARACTER_STREAM

```
22 BufferedReader open_character_stream(String fname) {  
23     BufferedReader br = null;  
24     if (fname == null) {  
25         br = new BufferedReader(new InputStreamReader(System.in));  
26     } else {  
27         try {  
28             FileReader fr = new FileReader(fname);  
29             br = new BufferedReader(fr);  
30         } catch (FileNotFoundException e) {  
31             System.out.print("The file " + fname + " doesn't exists\n");  
32             e.printStackTrace();  
33         }  
34     }  
35  
36     return br;  
37 }
```

Block Number	Lines	Entry	Exit	Function Calls
1	23, 24	23	24	
2	25	25	25	
3	28, 29, 36	28	36	



EDGE-COVERAGE

TEST PATHS:

```
[  
t1: (1, 2)  
t2: (1, 3)  
]
```

Unit Test

T1

- Input: String fname = "testfile.txt"
- Output: String "RANDOM STUFF // ##\n"

T2

- Input: String fname = ""
- Output: NULL

GET_TOKEN

```

94 String get_token(BufferedReader br)
95 {
96     int i=0,j;
97     int id=0;
98     int res = 0;
99     char ch = '\0';
100
101     StringBuilder sb = new StringBuilder();
102
103     try {
104         res = get_char(br);
105         if (res == -1) {
106             return null;
107         }
108         ch = (char)res;
109         while(ch==' ' || ch=='\n' || ch == '\r')
110         {
111             res = get_char(br);
112             ch = (char)res;
113         }
114
115         if(res == -1)return null;
116         sb.append(ch);
117         if(is_spec_symbol(ch)==true)return sb.toString();
118         if(ch=='"')id=2;    /* prepare for string */
119         if(ch=='#')id=1;    /* prepare for comment */
120
121         res = get_char(br);
122         if (res == -1) {
123             unget_char(ch,br);
124             return sb.toString();
125         }
126         ch = (char)res;
127
128         while (is_token_end(id,res) == false)/* until meet the end character */
129         {
130             sb.append(ch);
131             br.mark(4);
132             res = get_char(br);
133             if (res == -1) {
134                 break;
135             }
136             ch = (char)res;

```

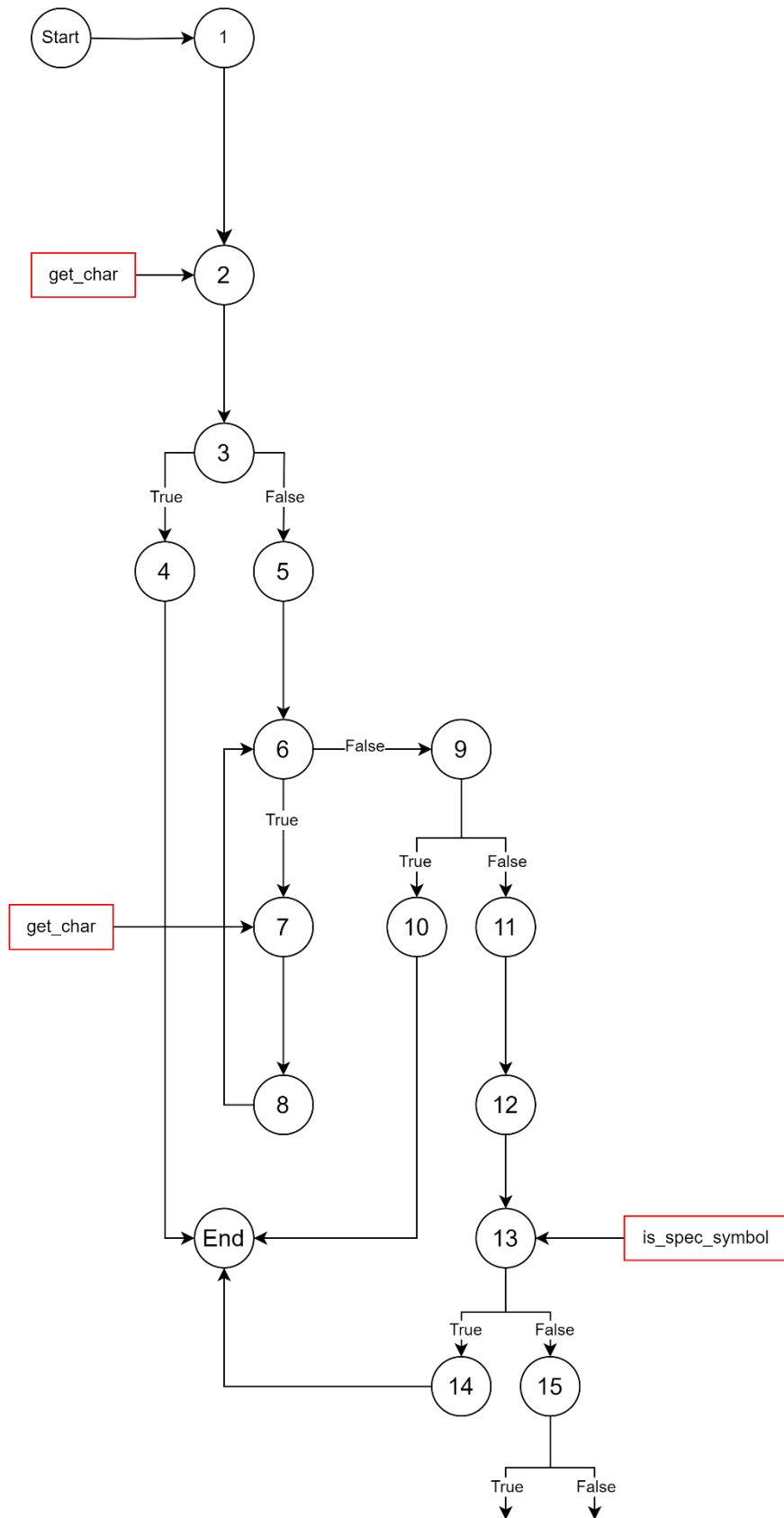
```

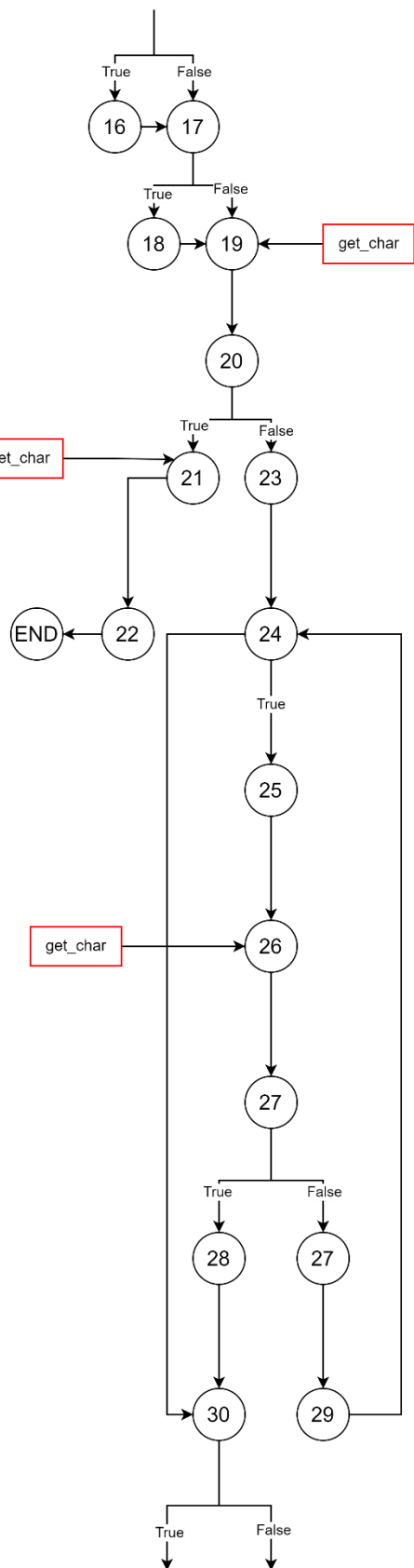
138
139     if(res == -1)          /* if end character is eof token */
140     { unget_char(ch,br);    /* then put back eof on token_stream */
141       return sb.toString();
142     }
143
144     if(is_spec_symbol(ch)==true) /* if end character is special_symbol */
145     { unget_char(ch,br);    /* then put back this character */
146       return sb.toString();
147     }
148     if(id==1)              /* if end character is " and is string */
149     {
150         if (ch == '"') {
151             sb.append(ch);
152         }
153         return sb.toString();
154     }
155     if(id==0 && ch==59)
156         /* when not in string or comment,meet ";" */
157         { unget_char(ch,br); /* then put back this character */
158           return sb.toString();
159         }
160 } catch (IOException e) {
161     e.printStackTrace();
162 }
163
164 return sb.toString();          /* return normal case token */
165 }

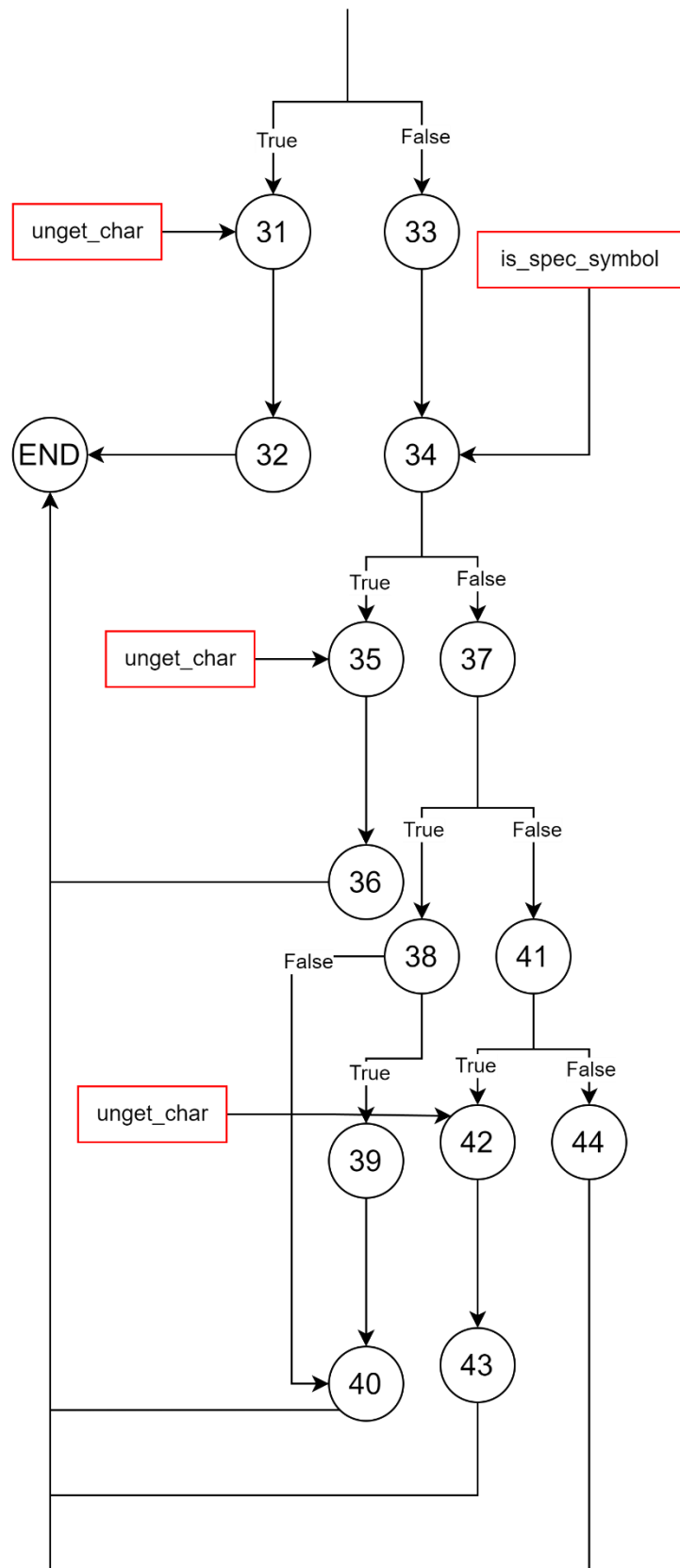
```

Block Number	Lines	Entry	Exit	Function Calls
1	96, 97, 98, 99, 101	96	101	
2	104	104	104	Get_char
3	105	105	105	
4	106	106	106	
5	108	108	108	
6	109	109	109	
7	111	111	111	Get_char
8	112	112	112	
9	115a	115a	115a	
10	115b	115b	115b	
11	116	116	116	
12	117a	117a	117a	
13	117b	117b	117b	is_spec_symbol
14	117c	117c	117c	
15	118a	118a	118a	
16	118b	118b	118b	
17	119a	119a	119a	
18	119b	119b	119b	
19	121	121	121	Get_char
20	122	122	122	
21	123	123	123	Unget_char
22	124	124	124	
23	126	126	126	

24	128	128	128	
25	130, 131	130	131	
26	132	132	132	Get_char
27	133	133	133	
28	134	134	134	
29	136	136	136	
30	139	139	139	
31	140	140	140	Unget_char
32	141	141	141	
33	144a	144a	144a	
34	144b	144b	144b	ls_spec_symbol
35	145	145	145	Unget_char
36	146	146	146	
37	148	148	148	
38	150	150	150	
39	151	151	151	
40	153	153	153	
41	155	155	155	
42	157	157	157	Unget_char
43	158	158	158	
44	164	164	164	







EDGE-COVERAGE

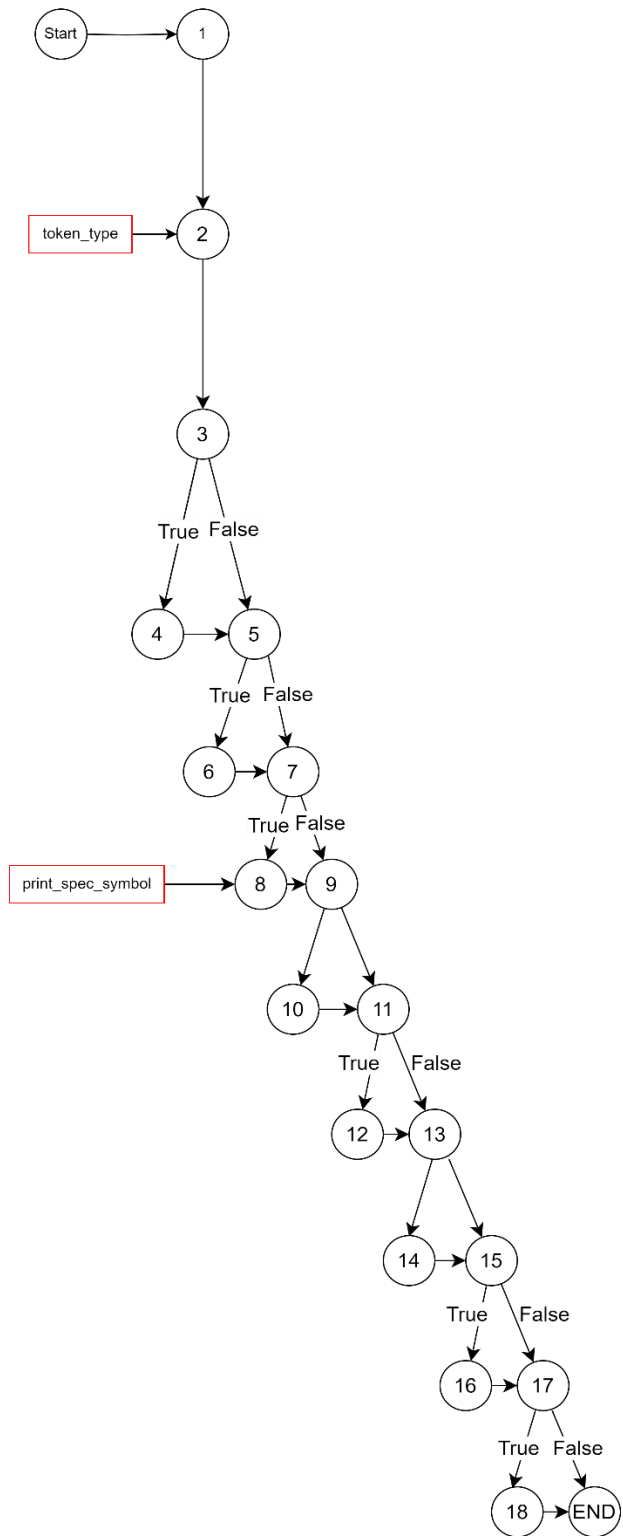
TEST PATHS:

```
[
(1, 2, 3, 4)
(1, 2, 3, 6, 7, 8, 6, 9, 10)
(1, 2, 3, 6, 9, 11, 12, 13, 14)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 24, 25, 26, 27, 29, 23, 24, 25, 26, 27, 28, 30, 31, 32)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 30, 33, 34, 35, 36)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 30, 33, 34, 37, 38, 40)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 30, 33, 34, 37, 38, 39, 40)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 30, 33, 34, 37, 41, 42, 43)
(1, 2, 3, 6, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 23, 30, 33, 34, 37, 41, 44)
]
```

PRINT_TOKEN

```
219 void print_token(String tok)
220 { int type;
221   type=token_type(tok);
222   if(type==error)
223   {
224     System.out.print("error,\"" + tok + "\".\n");
225   }
226
227   if(type==keyword)
228   {
229     System.out.print("keyword,\"" + tok + "\".\n");
230   }
231
232   if(type==spec_symbol)print_spec_symbol(tok);
233   if(type==identifier)
234   {
235     System.out.print("identifier,\"" + tok + "\".\n");
236   }
237   if(type==num_constant)
238   {
239     System.out.print("numeric," + tok + ".\n");
240   }
241   if(type==str_constant)
242   {
243     System.out.print("string," + tok + ".\n");
244   }
245   if(type==char_constant)
246   {
247     System.out.print("character,\"" + tok.charAt(1) + "\".\n");
248   }
249   if(type==comment)
250   {
251     System.out.print("comment,\"" + tok + "\".\n");
252   }
253 }
```

Block Number	Lines	Entry	Exit	Function Calls
1	220	220	220	
2	221	221	221	token_type
3	222	222	222	
4	224	224	224	
5	227	227	227	
6	229	229	229	
7	232a	232a	232a	
8	232b	232b	232b	print_spec_symbol
9	233	233	233	
10	235	235	235	
11	237	237	237	
12	239	239	239	
13	241	241	241	
14	243	243	243	
15	245	245	245	
16	246	246	246	
17	249	249	249	
18	251	251	251	



EDGE-COVERAGE

TEST PATHS:

```
[  
t1: (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18)  
t2: (1, 2, 3, 5, 7, 9, 11, 13, 15, 17)  
]
```

Unit Test

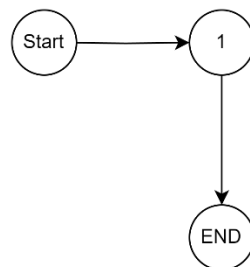
T1, T2

- Not semantically possible

GET_CHAR

```
39  /* ***** */  
40  /* NAME:    get_char          */  
41  /* INPUT:   a BufferedReader  */  
42  /* OUTPUT:  a character; when EOF, return -1  
43  /* ***** */  
44⊖ int get_char(BufferedReader br){  
45      int ch = 0;  
46      try {  
47          br.mark(4);  
48          ch= br.read();  
49      } catch (IOException e) {  
50          e.printStackTrace();  
51      }  
52      return ch;  
53  }  
54
```

Block Number	Lines	Entry	Exit	Function Calls
1	45, 47, 48, 52	45	52	



EDGE-COVERAGE

TEST PATHS:

```
[  
(1)  
]
```

Unit Test

T1

- Input: Bufferreader br = testfile.txt
- Output: int ch = 82

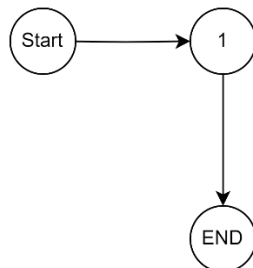
UNGET_CHAR

```

55      /*****
56      /* NAME:      unget_char
57      /* INPUT:      a BufferedReader,a character */
58      /* OUTPUT:     a character
59      /* DESCRIPTION:move backward.when unable to put back,return -1(EOF) */
60      /*****
61Ⓢ char unget_char (int ch,BufferedReader br) {
62          try {
63              br.reset();
64          } catch (IOException e) {
65              e.printStackTrace();
66          }
67          return 0;
68      }

```

Block Number	Lines	Entry	Exit	Function Calls
1	63, 67	63	67	



EDGE-COVERAGE

TEST PATHS:

```

[
(1)
]

```

Unit Test

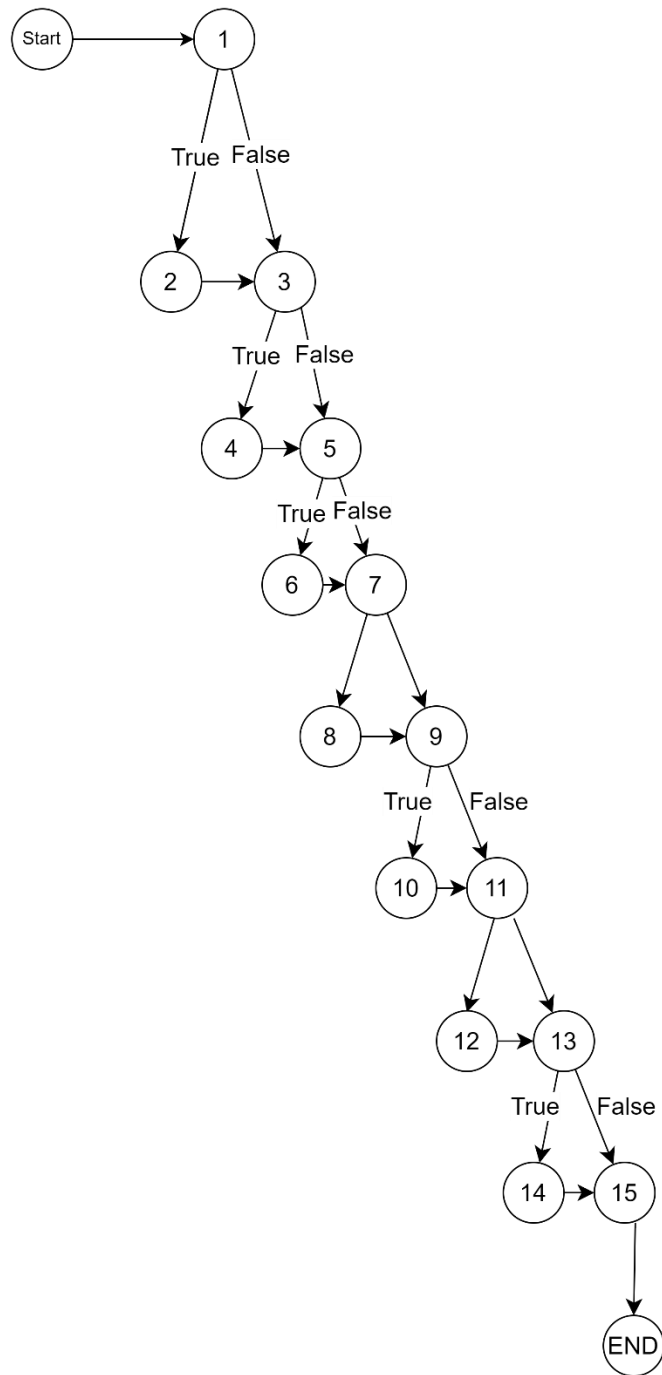
No available testing from JAR oracle file.

IS_SPEC_SYMBOL

```
420  /******  
421  /* NAME:      is_spec_symbol      */  
422  /* INPUT:     a token */  
423  /* OUTPUT:    a BOOLEAN value    */  
424  /******  
425  static boolean is_spec_symbol(char c)  
426  {  
427      if (c == '(')  
428      {  
429          return true;  
430      }  
431      if (c == ')')  
432      {  
433          return true;  
434      }  
435      if (c == '[')  
436      {  
437          return true;  
438      }  
439      if (c == ']')  
440      {  
441          return true;  
442      }  
443      if (c == '/')  
444      {  
445          return true;  
446      }  
447      if (c == '`')  
448      {  
449          return true;  
450      }  
451      if (c == ',')  
452      {  
453          return true;  
454      }  
455      return false;    /* others return FALSE */  
456  }
```

Block Number	Lines	Entry	Exit	Function Calls
1	427	427	427	
2	429	429	429	
3	431	431	431	
4	433	433	433	

5	435	435	435	
6	437	437	437	
7	439	439	439	
8	441	441	441	
9	443	443	443	
10	445	445	445	
11	447	447	447	
12	449	449	449	
13	451	451	451	
14	453	453	453	
15	455	455	455	



EDGE-COVERAGE

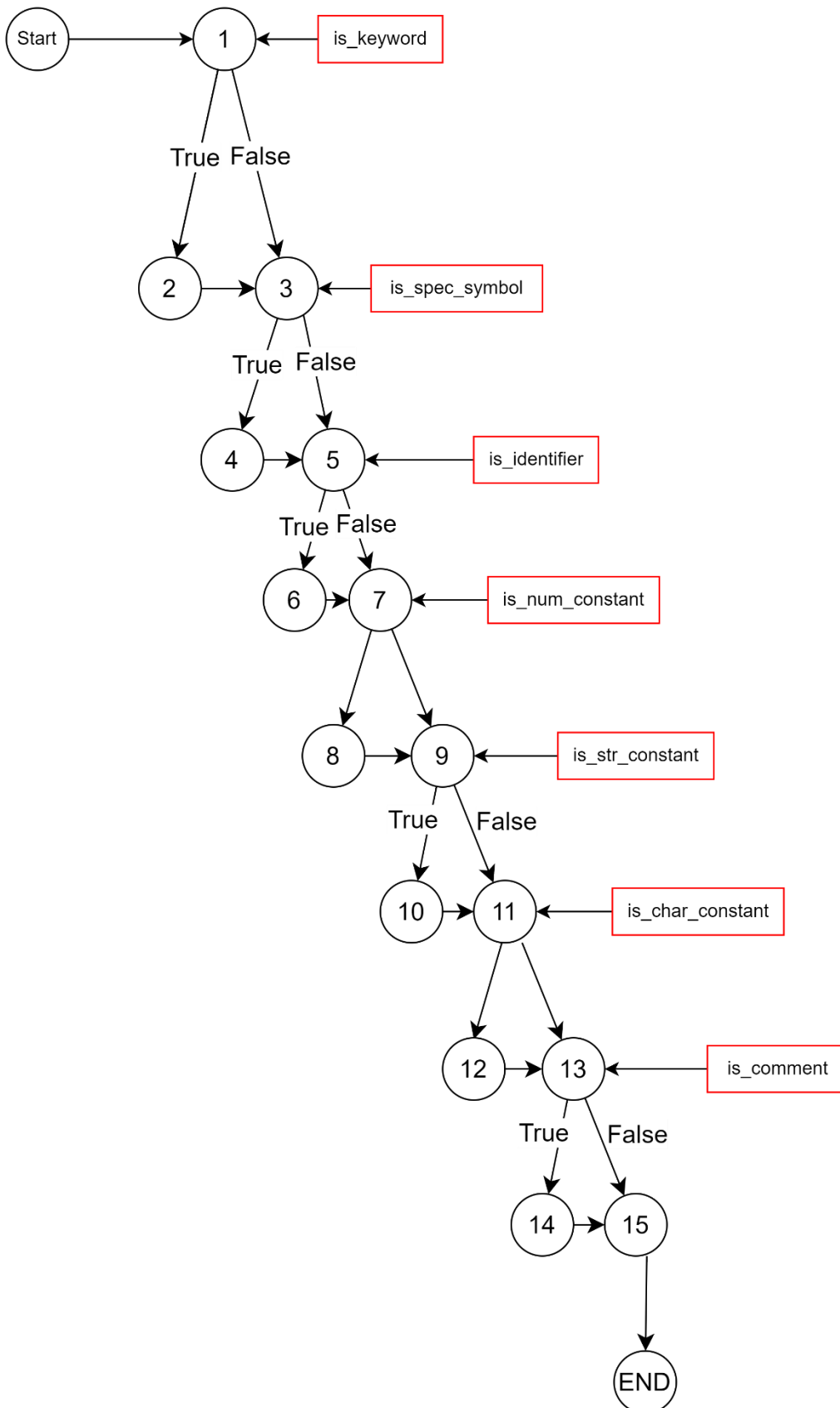
TEST PATHS:

```
[  
(1, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15  
(1, 2, 3, 5, 7, 9, 11, 13, 15  
]
```

TOKEN_TYPE

```
---  
196  /******  
197  /* NAME :   token_type  
198  /* INPUT:    a token  
199  /* OUTPUT:   an integer value  
200  /* DESCRIPTION: the integer value is corresponding  
201  /*           to the different token type  
202  /******  
203- static int token_type(String tok)  
204  {  
205    if(is_keyword(tok))return(keyword);  
206    if(is_spec_symbol(tok.charAt(0)))return(spec_symbol);  
207    if(is_identifier(tok))return(identifier);  
208    if(is_num_constant(tok))return(num_constant);  
209    if(is_str_constant(tok))return(str_constant);  
210    if(is_char_constant(tok))return(char_constant);  
211    if(is_comment(tok))return(comment);  
212    return(error);           /* else look as error token */  
213  }
```

Block Number	Lines	Entry	Exit	Function Calls
1	205a	205a	205a	Is_keyword
2	205b	205b	205b	
3	206a	206a	206a	Is_spec_symbol
4	206b	206b	206b	
5	207a	207a	207a	Is_identifier
6	207b	207b	207b	
7	208a	208a	208a	Is_num_constant
8	208b	208b	208b	
9	209a	209a	209a	Is_str_constant
10	209b	209b	209b	
11	210a	210a	210a	Is_char_constant
12	210b	210b	210b	
13	211a	211a	211a	Is_comment
14	211b	211b	211b	
15	212	212	212	



EDGE-COVERAGE

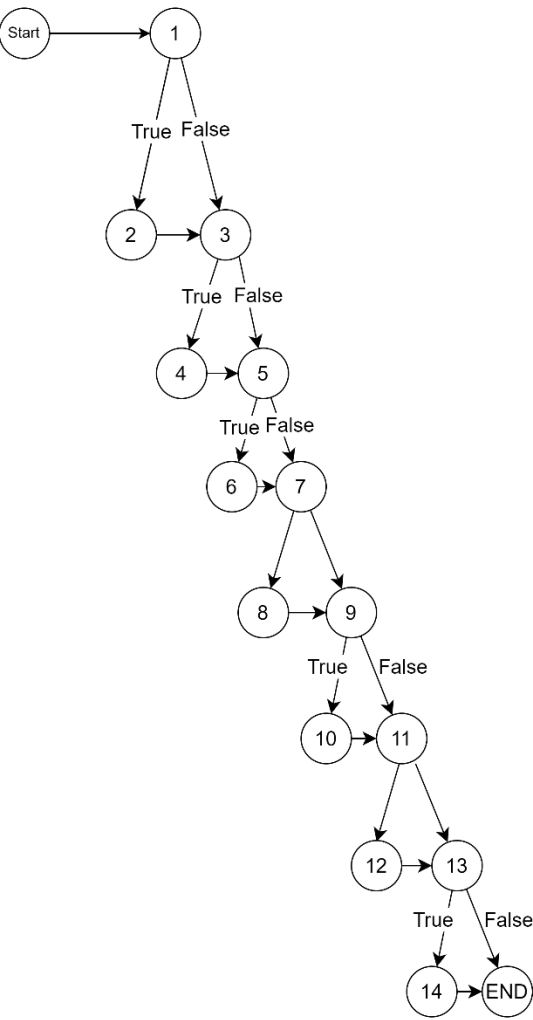
TEST PATHS:

```
[  
(1, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15)  
(1, 2, 3, 5, 7, 9, 11, 13, 15)  
]
```

PRINT_SPEC_SYMBOL

```
370  /******  
371  /* NAME:      print_spec_symbol          */  
372  /* INPUT:     a spec_symbol token */  
373  /* OUTPUT :   print out the spec_symbol token */  
374  /*          :   according to the form required */  
375  /******  
376  static void print_spec_symbol(String str)  
377  {  
378      if (str.equals("("))  
379      {  
380  
381          System.out.print("lparen.\n");  
382          return;  
383      }  
384      if (str.equals(")"))  
385      {  
386  
387          System.out.print("rparen.\n");  
388          return;  
389      }  
390      if (str.equals("[")  
391      {  
392          System.out.print("lsquare.\n");  
393          return;  
394      }  
395      if (str.equals("]")  
396      {  
397  
398          System.out.print("rsquare.\n");  
399          return;  
400      }  
401      if (str.equals("'")  
402      {  
403          System.out.print("quote.\n");  
404          return;  
405      }  
406      if (str.equals("`")  
407      {  
408  
409          System.out.print("bquote.\n");  
410          return;  
411      }  
412  
413      if (str.equals(",")  
414      {  
415          System.out.print("comma.\n");  
416          return;  
417      }  
418  }  
419
```

Block Number	Lines	Entry	Exit	Function Calls
1	378	378	378	
2	381, 382	381	382	
3	384	384	384	
4	387,388	387	388	
5	390	390	390	
6	392, 393	392	393	
7	395	395	395	
8	398, 399	398	399	
9	401	401	401	
10	403, 404	403	404	
11	406	406	406	
12	409, 410	409	410	
13	413	413	413	
14	415, 416	415	416	



EDGE-COVERAGE

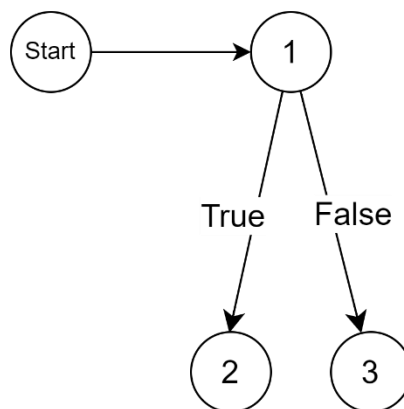
TEST PATHS:

```
[  
(1, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14)  
(1, 2, 3, 5, 7, 9, 11, 13)  
]
```

IS_KEYWORD

```
271  /******  
272  /* NAME:    is_keyword  
273  /* INPUT:   a token */  
274  /* OUTPUT:  a BOOLEAN value  
275  /******  
276  static boolean is_keyword(String str)  
277  {  
278      if (str.equals("and") || str.equals("or") || str.equals("if") ||  
279          str.equals("xor") || str.equals("lambda") || str.equals("=>"))  
280          return true;  
281      else  
282          return false;  
283  }  
284
```

Block Number	Lines	Entry	Exit	Function Calls
1	278	278	278	
2	280	280	280	
3	282	282	282	



EDGE-COVERAGE

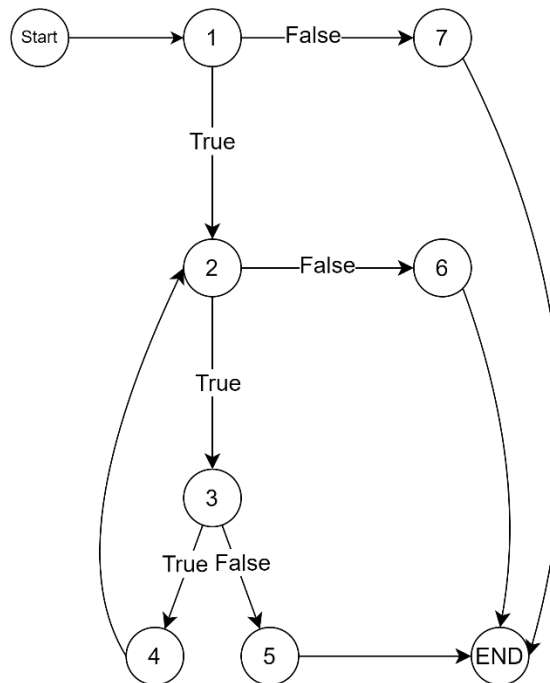
TEST PATHS:

```
[  
(1, 2)  
(1, 3)  
]
```

IS_IDENTIFIER

```
344  /******  
345  /* NAME:    is_identifier    */  
346  /* INPUT:   a token */  
347  /* OUTPUT:  a BOOLEAN value */  
348  /******  
349  static boolean is_identifier(String str)  
350  {  
351      int i=1;  
352  
353      if ( Character.isLetter(str.charAt(0)) )  
354      {  
355          while(i < str.length() && str.charAt(i) !='\0' ) /* unti meet the end token sign */  
356          {  
357              if(Character.isLetter(str.charAt(i)) || Character.isDigit(str.charAt(i)))  
358                  i++;  
359              else  
360                  return false;  
361          } /* end WHILE */  
362          return false;  
363      }  
364      else  
365          return true;  
366  }
```

Block Number	Lines	Entry	Exit	Function Calls
1	351, 353	351	353	
2	355	355	355	
3	357	357	357	
4	358	358	358	
5	360	360	360	
6	362	362	362	
7	365	365	365	



EDGE COVERAGE

TEST PATHS:

```

[
(1, 2, 3, 2, 6)
(1, 2, 3, 5)
(1, 7)
]

```

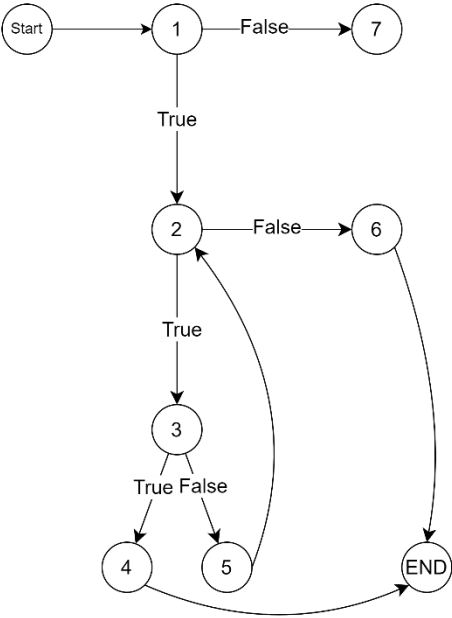
IS_STR_CONSTANT

```

322  /* ***** */
323  /* NAME:   is_str_constant   */
324  /* INPUT:  a token */
325  /* OUTPUT: a BOOLEAN value */
326  /* ***** */
327⊖ static boolean is_str_constant(String str)
328  {
329      int i=1;
330
331      if ( str.charAt(0) == '"' )
332          { while ( i < str.length() && str.charAt(i) != '\0' )
333              { if (str.charAt(i) == '"' )
334                  return true;          /* meet the second '"'
335                  else
336                      i++;
337              }          /* end WHILE */
338          return true;
339      }
340      else
341          return false;          /* other return FALSE */
342  }

```

Block Number	Lines	Entry	Exit	Function Calls
1	329, 331	329	331	
2	332	332	332	
3	333	333	333	
4	334	334	334	
5	336	336	336	
6	338	338	338	
7	341	341	341	



EDGE COVERAGE

TEST PATHS:

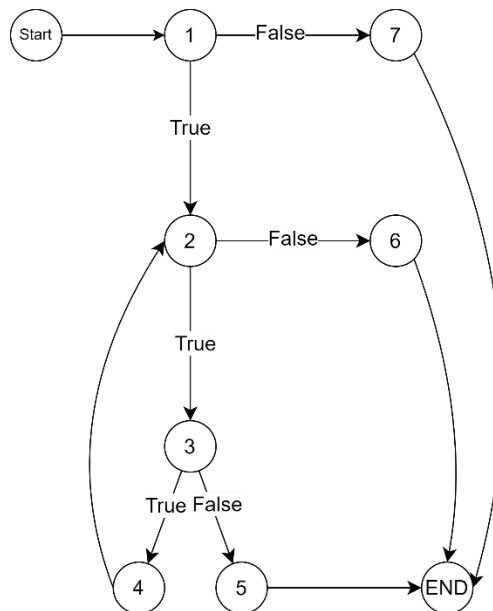
- [
- (1, 2, 3, 5, 2, 6)
- (1, 2, 3, 4)
- (1, 7)
-]

IS_NUM_CONSTANT

```

298  /*****
299  /* NAME:      is_num_constant      */
300  /* INPUT:     a token */
301  /* OUTPUT:    a BOOLEAN value    */
302  /*****
303  static boolean is_num_constant(String str)
304  {
305      int i=1;
306
307      if ( Character.isDigit(str.charAt(0)))
308      {
309          while ( i < str.length() && str.charAt(i) != '\0' )
310          {
311              if(Character.isDigit(str.charAt(i+1)))
312                  i++;
313              else
314                  return false;
315          }
316          return true;
317      }
318      else
319          return false;
320  }
  
```

Block Number	Lines	Entry	Exit	Function Calls
1	305, 307	305	307	
2	309	309	309	
3	311	311	311	
4	312	312	312	
5	314	314	314	
6	316	316	316	
7	319	319	319	



EDGE COVERAGE

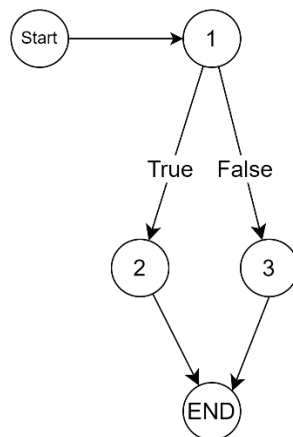
TEST PATHS:

```
[  
(1, 2, 3, 4, 2, 6)  
(1, 2, 3, 5)  
(1, 7)  
]
```

IS_CHAR_CONSTANT

```
285      /******  
286      /* NAME:    is_char_constant    */  
287      /* INPUT:   a token */  
288      /* OUTPUT:  a BOOLEAN value    */  
289      /******  
290  static boolean is_char_constant(String str)  
291  {  
292      if (str.length() > 2 || str.charAt(0)=='#' && Character.isLetter(str.charAt(1)))  
293          return true;  
294      else  
295          return false;  
296  }
```

Block Number	Lines	Entry	Exit	Function Calls
1	292	292	292	
2	293	293	293	
3	295	295	295	



EDGE COVERAGE

TEST PATHS:

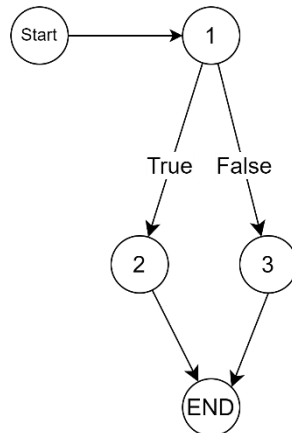
```
[  
(1, 3)  
]
```

(1, 2)
]

IS_COMMENT

```
258  /******  
259  /* NAME:    is_comment      */  
260  /* INPUT:   a token */  
261  /* OUTPUT:  a BOOLEAN value */  
262  /******  
263  static boolean is_comment(String ident)  
264  {  
265      if( ident.charAt(0) ==59 )  /* the char is 59  */  
266          return true;  
267      else  
268          return false;  
269  }
```

Block Number	Lines	Entry	Exit	Function Calls
1	265	265	265	
2	266	266	266	
3	268	268	268	



EDGE COVERAGE

TEST PATHS:

[
(1, 3)
(1, 2)
]